DAL24Circ og R4DS kap. 14 Strings - noter

Overblik

Dokumentet samler to ting til skolebrug: Wulfs video DAL24Circ i sammenhængende tekst og kapitel 14 (Strings) i R for Data Science. Videoen viser, hvordan man finder en regel for, om et fly cirkler eller flyver lige ud. Kapitlet giver værktøjerne til at arbejde med tekst i R, og dem bruger Wulf til URL'er, callsigns og filnavne.

Kapitel 14 dækker ikke regulære udtryk. Dem tager kapitel 15, og det er der `str_detect` ligger, som bruges til at finde SAS-fly i OpenSky-videoerne.

Uge 44 ligger bagerst, før forbehold og kilder: slides fra DALE26W44-API, øvelserne med løsningsforslag tre sider om DMI, lubridate og trafiktal og til sidst din Util.R og dit OpenSky-script med en fejlliste.

DAL24Circ: videoen i sammenhængende tekst

Videoen handler om at finde en regel, der kan afgøre, om et fly cirkler eller flyver lige ud. Wulf prøver to mål på otte træningsfly, han på forhånd ved cirkler: en lineær model og standardafvigelsen på kursen. Ingen af dem skiller flyene helt skarpt endnu.

Opsætning

Han starter fra det script, den forrige video sluttede med. Funktionen, der henter token, ligger i en separat `util.R`, som skal ligge samme sted som scriptet. Scriptet hedder `getcirc` version 1 og kører kun én gang nu. Senere skal det ligge i et `while`-loop.

Han bruger to endpoints. `states/all` giver alle fly i et område. `tracks` giver ruten for ét fly og skal bruge flyets `icao24` fra `states`. Bounding boxen skifter han fra Point Reyes til Nordsøen, fordi der næsten ikke er fly over Point Reyes. Derefter henter han token og laver GET-kald, tjekker at status er 200 og pakker JSON ud til en data frame med navngivne kolonner.

Træningsdata

De otte cirkelfly ligger som filer i mappen `train`. Han henter dem ved at bruge `list.files(path, pattern)` til at få en liste med filnavne og derefter `lapply` til at læse hver fil. Resultatet er en liste med otte data frames. En af dem kalder han `testdf`, og den har 352 observationer. Han sammenligner med Flightradar for at være sikker på, at data passer.

Plots til at udforske data

Han laver tre plots pr. fly ved siden af hinanden: ruten (lat mod lng), kurs mod tid og højde mod tid. Tiden ligger på y-aksen, så man følger udviklingen opad. For ikke at skrive det samme igen og igen pakker han plottene ind i en funktion, der tager en data frame og bruger `par` til at dele skærmen i tre.

Tracks for ét fly

Han tester med en tilfældig `icao24`. URL'en bygges af tracks-endpointet og flyets id. Status er 200, og `fromJSON` giver en liste, hvor ruten ligger som en matrix. Den gør han til en data frame med 63 observationer og giver kolonnerne navne. I praksis skal det være et loop, der henter track for hvert fly fra `states`. Tracks-endpointet giver kun flyets aktuelle rute, så de historiske træningsfly kan han ikke hente igen.

Kort til kontrol

For at se ruten bruger han `leaflet`: `addTiles` giver OpenStreetMap som baggrund, og `addPolylines` tegner ruten ud fra lat og lng. Første forsøg på træningsdata ser mærkeligt ud. Et normalt fly fra Norge til London ser rigtigt ud, så funktionen `myMapPlot` virker. Kortet bruges til at tjekke visuelt, om reglen rammer rigtigt.

Reglen: cirkler flyet?

Han tester to ideer på træningsdata. Den ene er en lineær model, `lm(lat ~ lng + crs + alt)`, kørt på et cirkelfly og på et normalt fly. Det normale fly får R^2 på 0,986 på skærmbilledet. De cirklende fly ligger ifølge transskriberingen mellem ca. 0,57 og 0,75, men de tal er læst fra en dårlig tekst. Modellen skiller altså, men ikke skarpt.

Den anden ide er standardafvigelsen på kursen, `sd(crs)`. Skærmbillederne viser 98,2 og 68,1 for to cirkelfly og 89,4 for det normale fly. Det normale fly ligger midt imellem, så `sd` skiller ikke. Wulf slutter med at lade `lm` stå som mål og skriver et skelet til sin egen detektionsfunktion, `myOwnDetectionAlg`, som han vil udforske videre.

R4DS kap. 14 Strings: funktioner

Kapitlet handler om at lave, sætte sammen, skille ad og måle på tekst med stringr og tidyr. Alle stringr-funktioner starter med `str_`.

Funktion	Hvad den gør	Eksempel
`str_view()`	Viser den rå streng uden escapes	`str_view(c("'", "\"", "\\"))`

Funktion	Hvad den gør	Eksempel
`r"(...)"`	Raw string, slipper for backslash-rod (R 4.0+)	`r"(\.csv$)"`
`str_c()`	Sætter strenge sammen, vektoriseret. Giver `NA`, hvis en del er `NA`	`str_c("Hi ", coalesce(name, "you"), "!")`
`str_glue()`	Udtryk i `{}` sættes ind i teksten. `NA` bliver til "NA"	`str_glue("Hi {name}!")`
`str_flatten()`	Samler en hel vektor til én streng, bruges i `summarize()`	`str_flatten(c("x", "y", "z"), ", ", last = ", and ")`
`separate_longer_delim()`	Deler en kolonne i nye rækker ved et skilletegn	`separate_longer_delim(x, delim = ",")`
`separate_wider_delim()`	Deler en kolonne i nye kolonner. Kolonnerne skal navngives	`separate_wider_delim(x, delim = "-", names = c("a", "b"))`
`separate_wider_position()`	Deler ved faste positioner	`widths = c(year = 4, age = 2, state = 2)`
`str_length()`	Tæller tegn	`str_length("EJU14BT")`
`str_sub()`	Klipper en bid ud. Negative tal tæller bagfra	`str_sub(x, 1, 3)` og `str_sub(x, -3, -1)`
`str_equal()`	Sammenligner tegn, der ser ens ud, men er kodet forskelligt	`str_equal(u[[1]], u[[2]])`

Tre ting er værd at huske. Ved `separate_wider_*` kan man bruge `too_few` og `too_many` med `"debug"` til at finde ud af, hvorfor en række ikke splitter pænt. `str_sub()` giver ikke fejl, hvis strengen er for kort. Og for tegn og sprog er UTF-8 standard: `locale(encoding = ...)` i readr bruges til andre encodings, og `guess_encoding()` er et første bud.

Sammenhæng: strings i OpenSky og DAL24Circ

Strings bruges fem steder i videoerne. Kapitel 14 dækker de fire første, og det sidste kræver kapitel 15.

Sted i videoerne	Hvad der sker	Kapitel 14-værktøj
Bygge URL'en	base url + endpoint + query bliver til en total url	`str_glue()` eller `str_c()`. Flere `icao24` kan samles med `str_flatten()`
Callsign har mellemrum bagest	`"EJU14BT "` har et mellemrum til sidst	`str_trim()` fra stringr, men den står ikke i kapitlets oversigt
Flyselskab ud af callsign	De tre første bogstaver er selskabet (SAS, EJU)	`str_sub(callsign, 1, 3)`. Giver SAS-filter uden regex
Indlæsning af filer og transskriberinger	`list.files(path, pattern)` bruger regulært udtryk. Æ, ø og å skal læses som UTF-8	Raw string `r"(\.csv$)"` og `locale(encoding = "UTF-8")`
Find SAS-fly med `str_detect`	`str_detect(callsign, "^SAS")` giver en logisk vektor	Kapitel 15, ikke 14

Bemærk, at Wulf søger på "SAS" hvor som helst i callsignet. `"^SAS"` kræver, at det starter med SAS, og er sikrere. Forskellen mellem hans 68 fly i R og 62 i terminalen er to snapshots på to tidspunkter og ikke en fejl, fordi `grep` tæller linjer og ikke fly.

Kode klar til noter

Koden samler det fra kapitel 14, der bruges i OpenSky-videoerne. `str_detect` til sidst er fra kapitel 15.

```
library(tidyverse)
library(httr)
library(jsonlite)

# 14.3: byg URL'en
base_url  <- "https://opensky-network.org/api"
endpoint  <- "states/all"
total_url <- str_glue("{base_url}/{endpoint}")

# Kald API'et og pak svaret ud
res      <- getRawFromUrl(total_url, getToken())   # kræver source("Util.R"),
```

```
se afsnittet om Util.R
status_code(res)   # 200 = ok
raw_list <- fromJSON(content(res, as = "text", encoding = "UTF-8"))
state_df <- as.data.frame(raw_list$states)
names(state_df) <- c("icao24", "callsign", "origin_country", "time_position",
                     "last_contact", "longitude", "latitude",
"baro_altitude",
                     "on_ground", "velocity", "true_track", "vertical_rate",
                     "sensors", "geo_altitude", "squawk", "spi",
"position_source")

# 14.5: ryd op i callsign og find selskab uden regex
state_df <- state_df |>
  mutate(callsign = str_trim(callsign),
         selskab  = str_sub(callsign, 1, 3))

state_df |> count(selskab, sort = TRUE)
sas_df <- state_df |> filter(selskab == "SAS")

# 14.2 + 14.6: raw string i mønster og UTF-8 ved indlæsning
filer <- list.files("train", pattern = r"(\.csv$)", full.names = TRUE)
tekst <- read_lines("DAL24Circ.txt", locale = locale(encoding = "UTF-8"))

# Kapitel 15: samme SAS-filter med regex
sas_df2 <- state_df |> filter(str_detect(callsign, "^SAS"))
```

To ting at passe på. `as.data.frame` på matrixen giver tekstkolonner, så brug `as.numeric()` på `latitude`, `longitude` og `velocity`, før du plotter. Wulf bruger brugernavn og kodeord i den første video, men dit eget setup bruger OAuth2 med client id og secret, se afsnittet om Util.R. Får du 401, er token udløbet, eller secret er forkert.

Uge 44: API-slides (DALE26W44)

Slides'ene følger tre trin: API tooling (Rscript, logning, `util.R`), OpenSky (endpoints, investigate, øvelse, Rscript) og OpenSky-casen (fly i bbox, andre API'er, cirklende fly). Det er samme rækkefølge som i Wulfs videoer, så slides er listen over det, du skal kunne forklare.

Baggrund

Big Data har fem V'er: Volume (mængde), Velocity (hastighed), Variety (forskellige formater og kilder), Veracity (usikkerhed og uens data) og Value (at trække noget brugbart ud). Et offentligt API gennemgår fem faser på slides: Investigate, Prototype, DataStore, Develop og Operations. Slides sætter også "Hidden" op som modstykke til

"Public". Min læsning er, at hidden API'er er dem, man finder ved at se, hvad en hjemmeside selv kalder i Chrome dev-tools. Det passer til DMI-siden senere i uge 44-materialet.

REST API og værktøjer

En request består af en URI og en metode (GET), og svaret kommer som JSON. Slides' punkter inden øvelsen er HTTP (kig i Chrome dev-tools), shell (Git Bash og `curl`) og R (`httr` + `jsonlite`).

I shell (Git Bash):

```
curl -s "https://opensky-network.org/api/states/all" | python -m json.tool
```

I R:

```
rurl <- "https://opensky-network.org/api"
res  <- httr::GET(totalurl, authenticate(user, pw))
```

OpenSky-endpoints

Rest API-menuen har syv punkter: All State Vectors, Own State Vectors, Flights in Time Interval, Flights by Aircraft, Arrivals by Airport, Departures by Airport og Track by Aircraft. Slides viser stierne i tabellen.

Sti	Hvad den giver	Parametre
`/states/all`	Alle fly lige nu	`time` (unix-sekunder, udeladt = nu), `icao24` (hex, flere ved at gentage parameteren), `lamin`, `lomin`, `lamax`, `lomax` til bounding box
`/flights/aircraft`	Flyvninger for ét fly	`icao24` (kun små bogstaver), `begin`, `end` (unix-sekunder). Intervallet må højst være 30 dage
`/flights/all`	Flyvninger i et tidsinterval	Vises kun som demo-punkt
`/flights/arrival`	Ankomster til en lufthavn	Vises kun som demo-punkt
`/tracks/all`	Rute for ét fly	`icao24`, `time=0`. Eksempel: `tracks/all?icao24=3c4b26&time=0`

Demoen i slides er: 1) registrér dig på tjenesten, 2) beskriv API'ets endpoints, først overordnet og derefter i detaljer.

Tidsstempler

Svaret bruger unix-tid, altså sekunder siden 1. januar 1970. Slides spørger "Hvad er det?" ved de to tal i en state. Det er `time_position` og `last_contact`, kolonne 4 og 5. Tallet 1664437945 i eksemplet er 29. september 2022 kl. 07:52:25 UTC. I R laver du tallet om med `as.POSIXct(timestamp, origin = "1970-01-01", tz = "UTC")`, og nutiden som unix-tal får du med `as.integer(Sys.time())`.

R-opgaver og andre API'er

R-opgaverne på slides er: source et script med en funktion (`getToken`), logning med `logr`, `while`-loop, `Rscript`, `tryCatch`, kommandolinje-argumenter og konvertering mellem unix-tid og dato. Under "Andre API'er" viser slides fire eksempler: Salling Group (der skal man registrere sig til Foodwaste), Rejseplanen Labs (en request om adgang), Datafordeleren (API-keys med udløbsdato) og watsonC (synkroniserer data med Jupiter og Danmarks Miljøportal).

Uge 44: øvelser og caser

Slides har seks OpenSky-øvelser og to cases: Point Reyes og et cirklende fly. Tabellen siger, hvad hver øvelse kræver, og hvordan jeg ville løse den i R. Tidspunkterne er regnet efter, men API-kald og kode er ikke kørt.

#	Opgave	Løsning eller hint
1	Hvornår fejrer man 1234567890-dagen?	13. februar 2009 kl. 23:31:30 UTC, altså 14. februar 2009 kl. 00:31:30 dansk tid. `as.POSIXct(1234567890, origin = "1970-01-01", tz = "UTC")`
2	Hvad skete der 2017-07-14 04:40:00 CEST?	Tidspunktet er præcis unix-tid 1500000000, altså 1,5 milliarder sekunder (02:40 UTC). Det er endnu en rund unix-dag

#	Opgave	Løsning eller hint
3	Alle fly i luften nu: hvor mange Ryanair, hvilket land har flest, barplot over top 10	Hent `states/all`. Ryanair: `str_detect(callsign, "^RYR")`. Land: `count(origin_country, sort = TRUE)` og `head(10)` til barplot. `origin_country` er det land, flyets adresse er registreret i, ikke hvor det flyver
4	Find et flys icao24 på Flightradar24 og slå det op	Flightradar viser "ICAO 24-bit address" i flyets infoboks (på slide: MMF21 med 480C42). Brug det med små bogstaver i `states/all?icao24=480c42`
5	Funktion `getFlight(icao, start, slut)`. Vis at Taylor Swift fløj i sit privatfly 1. april 2021	Eksemplet `getFlight("896174", 1730013733, 1730158120)` er 27. oktober 2024 kl. 07:22:13 UTC til 28. oktober kl. 23:28:40 UTC, 1,7 døgn og dermed under 30-dages grænsen. Til Taylor Swift skal du finde hendes flys icao24 (opgave 4) og regne datoen om til unix-tid. Det har jeg ikke slået op
6	Læg funktionen i et script, der køres som `Rscript getF.R "896174" 1730013733 1730158120`	`commandArgs(trailingOnly = TRUE)` giver argumenterne som tekst

Koden til opgave 5 og 6 forudsætter, at `getToken()` i `util.R` returnerer selve token-strengen, og at OpenSky tager det som Bearer token. Det passer med din egen `util.R`.

```
library(tidyverse)
library(httr)
library(jsonlite)
source("util.R")   # indeholder getToken()

getFlight <- function(icao, start, slut) {
  url <- str_glue("https://opensky-network.org/api/flights/aircraft",
                  "?icao24={icao}&begin={start}&end={slut}")
  res <- GET(url, add_headers(Authorization = paste("Bearer", getToken())))
  stop_for_status(res)
  fromJSON(content(res, as = "text", encoding = "UTF-8"))
}
```

Scriptet `getF.R` til opgave 6:

```
#!/usr/bin/env Rscript
source("util.R")
args <- commandArgs(trailingOnly = TRUE)
res  <- getFlight(args[1], as.integer(args[2]), as.integer(args[3]))
print(res)
```

Kommandoen kører du i PowerShell, når du står i den mappe, hvor `getF.R` ligger: `Rscript getF.R "896174" 1730013733 1730158120`.

Case: Point Reyes

Spørgsmålet er, hvor mange fly der flyver over Point Reyes i løbet af et minut, og i løbet af tre døgn. Til tre døgn skal scriptet gemme data frame i en fil og køre i konsollen eller terminalen. Til dagen efter skal I finde jeres eget område og sætte det op i et script.

Bounding boxen fra slides er `lamin` 37.841183, `lamax` 38.228414, `lomin` -123.143884 og `lomax` -122.573969. Pas på rækkefølgen: slidens rå tal står som lon, lat, lon, lat (-123.14, 37.84, -122.57, 38.23), men OpenSky vil have `lamin`, `lomin`, `lamax`, `lomax`. Ombyttes bredde og længde, får du fly et helt andet sted.

```
kolonner <- c("icao24", "callsign", "origin_country", "time_position",
              "last_contact", "longitude", "latitude", "baro_altitude",
              "on_ground", "velocity", "true_track", "vertical_rate",
              "sensors", "geo_altitude", "squawk", "spi", "position_source")

bbox_url <- str_glue("https://opensky-network.org/api/states/all",
                     "?lamin=37.841183&lomin=-123.143884",
                     "&lamax=38.228414&lomax=-122.573969")

slut  <- Sys.time() + 60        # 1 minut. Til 3 døgn: 3 * 24 * 3600
liste <- list()

while (Sys.time() < slut) {
  res <- tryCatch(
    GET(bbox_url, add_headers(Authorization = paste("Bearer", getToken()))),
    error = function(e) NULL)
  if (!is.null(res) && status_code(res) == 200) {
    raw <- fromJSON(content(res, as = "text", encoding = "UTF-8"))
    if (!is.null(raw$states)) {          # tomt område giver NULL
      df <- as.data.frame(raw$states)
      names(df) <- kolonner
      df$hentet <- raw$time
      liste[[length(liste) + 1]] <- df
    }
  }
```

```
  Sys.sleep(10)                              # skån rate limit
}

totaldf <- do.call("rbind", liste)           # som på slide
saveRDS(totaldf, "pointreyes.rds")
length(unique(totaldf$icao24))               # antal forskellige fly
```

Slides' barplot viser antal fly pr. hentning. Datoerne på x-aksen laver du med `as.POSIXct(as.numeric(df$time_position), origin = "1970-01-01")`.

Case: cirklende NATO-fly

Slides' ni trin er de samme, som Wulf går igennem i DAL24Circ. Find eksemplet med bbox for `/states/all` (Schweiz), find en boundingbox over et område I vil overvåge (Østersøen), og ændr Schweiz-URL'en til jeres område. Gem `icao24`-numrene i en vektor, og hent tracks med `/tracks/all` for to fly, hvor det ene cirkler og det andet flyver lige ud. Derefter skal I finde mål, der kan spotte et cirklende fly: scatterplot `lat ~ long`, lineplot med tid mod kurs, `sd(kurs)` og en egen funktion `MyOwnDetectionFunction(lat, long)`. Til sidst tæller I fly over boksen i 3 minutter, gemmer data frame i en fil og kører det hele i konsollen eller terminalen.

Slidens to plots viser forskellen. Det ene er en næsten ret linje, mens det andet er krøllet. Det er samme ide som Wulfs `lm` og standardafvigelse på kursen: et lige fly passer til en linje, et cirklende gør ikke.

Uge 44: DMI, lubridate og trafiktal

De tre sider i det andet bilag er tre værktøjer, der passer til ugens tema: at finde API-kaldet bag en hjemmeside, at regne med tid og at læse en trafikrapport.

DMI og Chrome dev-tools

Skærmbilledet viser DMI's side for observationer (`dmi.dk/friedata/observationer`) med Chrome dev-tools åben på fanen Network, filtreret på Fetch/XHR. Her kan man se, hvilke kald siden selv laver: `items?status=Active&datetime=2026-10-22T08:06:30.398Z`, kaldt fra `LoadOpendataMap.js` med status 200. Teknikken er at finde det kald, kopiere URL'en og prøve den i R. Det er den samme tankegang som Wulf bruger, når han undersøger et API, før han koder.

lubridate: hent og ændr dele af en dato

Arket "Get and set components" viser, hvordan man trækker enkelte dele ud af et tidspunkt. Det er praktisk, når du har lavet unix-tid om til dato og vil tælle fly pr. time eller dag.

Funktion	Giver
`date(x)`	Datoen
`year(x)`, `month(x)`, `day(x)`	År, måned, dag i måneden
`wday(x, label, abbr)`	Ugedag
`hour(x)`, `minute(x)`, `second(x)`	Klokkeslet i dele
`tz(x)`	Tidszone
`week(x)`, `isoweek()`, `epiweek()`	Ugenummer
`quarter(x)`, `semester(x)`	Kvartal og halvår
`am(x)`, `pm(x)`, `dst(x)`, `leap_year(x)`	Formiddag, eftermiddag, sommertid, skudår
`update(x, mday = 2, hour = 1)`	Ændrer dele i et tidspunkt

`as_datetime(1664437945)` fra lubridate laver unix-tid om til dato med 1970 som udgangspunkt. Til at samle fly pr. time bruger du `floor_date(tid, "hour")`. Den står ikke på arket, men er også lubridate.

Trafiktal fra Vejdirektoratet

Siden er en Excel-rapport fra Vejdirektoratets Mastra: Hastighedsklassifikation, døgnoversigt for målested TSA 20 Holbæk V (MV afslutning), Holbækmotorvejen (vej 11-0), retning mod København, motorkøretøjer, 1. marts 2026, hastighedsgrænse 130 km/t. Rækkerne er kvarter for kvarter, og kolonnerne er hastighedsintervaller fra 0-60 op til 160-180 km/t med totaler og gennemsnitshastighed. De første rækker er fx 19 køretøjer mellem 00:00 og 00:15 med gennemsnit 101,8 km/t. Kortet ved siden af viser tællestationer med årsdøgnstrafik (ADT), fx 25.723 på stationen ved Holbækmotorvejen i 2026. Rapporten bærer en note om, at rapporter over 100 sider skal begrænses.

I R kan `readxl::read_excel(fil, skip = 20)` springe de første 20 rækker over, så tabellen starter ved overskrifterne. Tjek rækkenummeret mod din egen fil. Det ligner trafiktal-filen til fredagens opgave, men det ved jeg ikke.

OpenSky finalboss: Util.R og scriptet

`Util.R` henter et OAuth2-token og kalder API'et med det. Scriptet bruger den til at finde SAS-fly, joine operatører og lufthavne, hente afgange fra Rom og tegne en rute over Den Persiske Golf. Det virker i store træk, men fem ting gør, at det fejler, hvis man kører det oppefra.

Util.R

`getToken()` sender en POST til OpenSkys auth-server med `grant_type = "client_credentials"`, client id og secret og returnerer `access_token`, hvis status er 200. Ellers returnerer den `NULL`. `getRawFromUrl()` sender en GET med headeren `Authorization: Bearer <token>`. Det bekræfter, at `getToken()` returnerer selve token-strengen, som `getFlight` i øvelsesafsnittet forudsætter. Secret er fjernet med vilje. Den skal heller ikke stå i filen, når den ligger i det fælles repo.

Tre ting i `Util.R` er svage:

- 401-grenen i `getRawFromUrl()` henter et nyt token, men sender ikke kaldet igen og returnerer det gamle svar. Et udløbet token giver derfor stadig 401.
- Fejler `getToken()`, får du `NULL`, og kaldet går af sted med "Bearer" uden token, uden at der kommer en klar fejl.
- Scriptet overskriver `getRawFromUrl()` med en egen version (se tabellen nedenfor).

En rettet version, hvor secret ligger i `.Renviron`. Det åbner du i RStudio-konsollen med `usethis::edit_r_environ()`, og så skriver du `OPENSKY_SECRET=din_secret` som en linje:

```
library(httr)
library(jsonlite)
library(dplyr)

clientId     <- "casperstegler@gmail.com-api-client"
clientSecret <- Sys.getenv("OPENSKY_SECRET")   # ligger i .Renviron, ikke i
filen

getToken <- function() {
  response <- POST(
    url = "https://auth.opensky-network.org/auth/realms/opensky-
network/protocol/openid-connect/token",
    body = list(grant_type = "client_credentials",
                client_id = clientId,
                client_secret = clientSecret),
    encode = "form")
  stop_for_status(response)
  content(response, as = "parsed", type = "application/json")$access_token
```

```
}

getRawFromUrl <- function(url, token) {
  res <- GET(url, add_headers(Authorization = paste("Bearer", token)))
  if (status_code(res) == 401) {      # token udløbet: hent nyt og prøv én
gang til
    res <- GET(url, add_headers(Authorization = paste("Bearer", getToken())))
  }
  res
}
```

I scriptet henter du så token én gang med `token <- getToken()` og bruger `getRawFromUrl(toturl, token)`. Så henter du ikke et nyt token ved hvert kald.

Scriptet trin for trin

Scriptet går igennem syv trin. Først bygger det URL'en med `str_c(baseurl, endpoint)`. Derefter henter det `/states/all`, pakker JSON ud med `fromJSON` og gør `$states` til en data frame. Kolonnenavnene scraper det med `rvest` fra OpenSkys dokumentation (tabel 5), dropper den sidste række (`category`, som kun kommer med i det udvidede svar) og gemmer dem som `vars.rds`.

Dernæst finder det operatøren i callsignet med `str_extract(callsign, "[A-Z]+")`, som tager det første stykke store bogstaver: SAS646 giver SAS, N412Z giver N. FAA's liste over operatørkoder med tre bogstaver scraper det med `html_elements()` og `html_table()` og gemmer som `operators.rds`. Lufthavne kommer fra `airports.csv`, og landekoder scraper det fra IBAN's tabel.

Til sidst henter det afgange fra Rom med `/flights/departure?airport=LIRF&begin=...&end=...`. Tidsstemplerne ligger et døgn tilbage, fordi flights-endpoints, så vidt jeg ved, først opdateres natten efter. Og det henter track for ét fly over Den Persiske Golf med `/tracks/all?icao24=...&time=0`, giver de ti kolonner navne, filtrerer på `alt > 6000`, plotter kurs mod tid, tager `sd(crs)` og tegner ruten på et kort med `rnaturalearth`.

Fem ting, der får scriptet til at fejle

Linje	Problem	Løsning
43	`rawres` bruges, men linjen der laver den (36) er udkommenteret	Fjern `#` på `rawres <- getRawFromUrl(toturl, token)`

Linje	Problem	Løsning
29-34	Scriptet overskriver `getRawFromUrl()` fra `Util.R` med en version, der henter nyt token ved hvert kald	Slet den og brug den fra `Util.R`
79	`left_join(tabsdf)` uden `by`. `currentDF` har `cslet`, `tabsdf` har `operator`, og så vidt jeg kan se ingen fælles kolonner	`left_join(tabsdf, by = c("cslet" = "operator"))`
86	Stien `/Users/vandmelonsguru/...` er en Mac-sti, som ikke findes på din Windows-maskine	Brug en relativ sti som til `Util.R`, fx `read.csv("Fælles/November/Opensky/airports.csv")`
144, 147, 154	`tykdf` og `tykdfH` findes ikke, kun `tykdf2` og `tykdfH2`	Skift til `tykdf2` og `tykdfH2`

Småting: `icao24` får to værdier efter hinanden, og den første bliver overskrevet. `names` overskriver R's egen funktion med det navn, så kald den fx `kolonnenavne`. Og IBAN-joinet er ikke nødvendigt for at finde Italien, for `airports.csv` har allerede `iso_country`, så `filter(iso_country == "IT")` er nok.

Svaghed i standardafvigelse på kursen

Kurs er en vinkel. 359° og 1° ligger tæt på hinanden, men `sd()` ser dem som langt fra hinanden. Et fly, der flyver lige, men krydser nord, får derfor høj standardafvigelse og ligner et cirklende fly. Et bedre mål er den samlede drejning, altså hvor mange grader kursen ændrer sig i alt. En runde giver omkring 360 grader, og et lige fly giver næsten 0. Det er mit forslag og ikke noget fra kurset, så tjek, om Wulf accepterer det.

```
drejning <- function(crs) {
  d <- diff(crs)
  d <- (d + 180) %% 360 - 180      # ændring i grader, mellem -180 og 180
  sum(abs(d), na.rm = TRUE)
}

drejning(tykdfH2$crs)
```

Wulfs getCircl.R: samlet kode og forklaring

Koden er sat sammen ud fra skærmbilleder af Wulfs RStudio i den rækkefølge, han skriver den. Jeg har flyttet de to funktioner op, så filen kan køres oppefra, for Wulf kører linje for linje. Wulfs client secret står synlig i Environment-panelet på flere skærmbilleder. Den er udeladt, og den skal du ikke kopiere. Selve secret og client id ligger i `util.R`.

Samlet kode

```
library(jsonlite)
library(httr)
library(stringr)
library(leaflet)
source("util.R")                        # getToken()

baseurl = "https://opensky-network.org/api"

endpointS = "/states/all"
endpointT = "/tracks/all"

queryS = paste0(baseurl, endpointS)
queryT = paste0(baseurl, endpointT)

# hjælpefunktioner
myMapPlot <- function(df) {
  leaflet(df) %>%
    addTiles() %>%
    addPolylines(lat = ~lat, lng = ~lng)
}

myEDAPlots <- function(df) {
  par(mfcol = c(1, 3), ann = F)
  plot(df$lat, df$lng)
  plot(df$crs, df$time)
  plot(df$alt, df$time)
}

#### NORTH SEA ####
lamin = 52.055129
lamax = 56.196869
lomin = -6.065140
lomax = 4.305954

fullurl = paste0(baseurl, endpointS, "?lamin=", lamin, "&lomin=", lomin,
"&lamax=", lamax, "&lomax=", lomax)
```

```
# one take over NS (skateboard)
token = getToken()
res <- httr::GET(fullurl, add_headers(Authorization = paste("Bearer",
token)))
rescontent <- httr::content(res, as = "text")
resretval <- jsonlite::fromJSON(rescontent)
statedfpr <- as.data.frame(resretval$states)

# find all icaos and put in vector
icaov = statedfpr$V1

# find all tracks for each icao (her kun ét fly)
ictest = icaov[[6]]
token = getToken()
turl = paste0(baseurl, endpointT, "?icao24=", ictest, "&time=0")
res <- httr::GET(turl, add_headers(Authorization = paste("Bearer", token)))
res$status_code
resretval <- fromJSON(content(res, as = "text"))
statedf <- as.data.frame(resretval)
colnv = c("icao24", "callsign", "startTime", "endTime", "time", "lat", "lng",
"alt", "crs", "grd")
colnames(statedf) = colnv
myEDAPlots(statedf)

# load training-data
fl = list.files(path = "./train", pattern = "*rds", full.names = T)
collist = lapply(fl, readRDS)
testdf = collist[[4]]
testdf = readRDS("circMT.rds")

# plots to explore data
myEDAPlots(testdf)
# map-plot to verify flight
myMapPlot(testdf)

# apply circmetrics
# lm
cmodel = lm(lat ~ lng + crs + alt, data = testdf)
nmodel = lm(lat ~ lng + crs + alt, data = statedf)
summary(cmodel)
summary(nmodel)
# sd
csd = sd(testdf$crs)
nsd = sd(statedf$crs)

# myOwnDetectionAlg
```

I Wulfs første version af URL'en står hele adressen skrevet ud i `fullurl`. Han erstatter den med `paste0(baseurl, endpointS, ...)`, som står her.

Forklaring blok for blok

Kode	Hvad den gør	Kapitel 14 og videoen
`paste0(baseurl, endpointS)`	Bygger URL'en af base og endpoint	Videoen: "base url + endpoint + query = total url". Kapitel 14.3: `str_c()` gør det samme, men giver `NA`, hvis en del er `NA`, mens `paste0()` skriver "NA" ind i teksten. `stringr` er loadet i scriptet, men bruges ikke
`lamin`, `lomin`, `lamax`, `lomax` og `fullurl`	Bounding box om Nordsøen og en query efter `?`	Videoen: Point Reyes er skiftet ud, fordi der ikke er mange fly der. Rækkefølgen i URL'en er `lamin`, `lomin`, `lamax`, `lomax`
`GET`, `content(as = "text")`, `fromJSON`, `as.data.frame(resretval$states)`	Henter, pakker JSON ud og laver data frame	`$states` er en matrix uden kolonnenavne, så kolonnerne hedder `V1` til `V17`. Derfor er `statedfpr$V1` icao24. `rescontent` er en lang streng, og i Environment vises den med backslashes, fx `"{\"icao24\":...`. Det er 14.2.1: `print` viser escapes, `str_view()` eller `cat()` viser indholdet, som det er. JSON selv er kapitel 23, ikke 14

Kode	Hvad den gør	Kapitel 14 og videoen
`icaov[[6]]` og `turl`	Tager det sjette fly og henter dets track	Videoen: "normalt er det jo et loop, så nu tager vi bare en random icao". `&time=0` giver flyets aktuelle rute
`colnv` og `colnames()`	Navngiver de ti kolonner i track	Track giver `icao24`, `callsign`, `startTime`, `endTime` og seks kolonner fra `path`: `time`, `lat`, `lng`, `alt`, `crs`, `grd`
`list.files(pattern = "*rds")` og `lapply(fl, readRDS)`	Finder otte gemte cirkelfly og læser dem ind i en liste	`pattern` er et regulært udtryk og ikke et filnavnsmønster. `"*rds"` virker, men rammer alt, der indeholder "rds". Korrekt er `r"(\.rds$)"`, en raw string (14.2.2)
`myEDAPlots()` og `myMapPlot()`	Tre plots ved siden af hinanden og et kort	Videoen: "så hedder den ikke test, så hedder den df". Første version brugte `testdf` inde i funktionen i stedet for argumentet `df`. `par(mfcol = c(1, 3), ann = F)` deler skærmen i tre, og tiden ligger på y-aksen
`lm(lat ~ lng + crs + alt)`	Model på et cirkelfly (`cmodel`) og et normalt fly (`nmodel`)	Normalt fly får R^2 på 0,986 på skærmbilledet. De cirklende fly ligger ifølge transskriberingen mellem 0,57 og 0,75

Kode	Hvad den gør	Kapitel 14 og videoen
`sd(crs)`	Standardafvigelse på kurs	Skærmbillederne viser 98,2 og 68,1 for to cirkelfly og 89,4 for det normale fly

Resultatet er det vigtigste, Wulf selv ender på. Standardafvigelsen skiller ikke cirkelfly fra et normalt fly, for det normale fly ligger midt imellem. `lm` skiller bedre, men ikke skarpt. Det er derfor, han slutter med et skelet til sin egen detektionsfunktion, `myOwnDetectionAlg`. Her passer `drejning()` fra afsnittet om Util.R, der summerer kursændringerne.

Hvor kapitel 14 kan gøre koden bedre

To ændringer, der hører til kapitlet. URL'en kan bygges med `str_glue`, og filnavnsmønsteret kan skrives som raw string:

```
fullurl <- str_glue("{baseurl}{endpointS}?lamin={lamin}&lomin={lomin}&lamax=
{lamax}&lomax={lomax}")
fl <- list.files("./train", pattern = r"(\.rds$)", full.names = TRUE)
```

Til callsign fra `states/all` giver `str_trim()` og `str_sub(callsign, 1, 3)` flyselskabet, som i afsnittet om strings.

Forbehold og kilder

Transskriberingen af DAL24Circ er støjende. Der er stykker, hvor Whisper bare gentager "DA", "Ro" og "Du", og dem er ignoreret. Hvilke variable `lm` kører på, er et bedste gæt ud fra, at han plotter lat mod lon og tid mod kurs. R^2-tallene er aflæst fra en dårlig tekst, så tjek dem i videoen omkring 22:00 til 26:00, hvis de skal bruges i en opgave. Oversigten over kapitel 14 er hentet fra nettet og ikke læst fra den lokale HTML-fil.

Uge 44-afsnittene er læst fra slides og skærmbilleder. Tidspunkter er regnet efter, men API-kald og kode er ikke kørt, og jeg har ikke slået Taylor Swifts flys icao24 op. Slidens tekst om Hidden API er min egen læsning. Fejllisten for Util.R og scriptet er læst fra koden og ikke kørt.

Kilder

- R for Data Science (2e), kapitel 14: Strings
- Transskribering af DAL24Circ (Wulf, DALE24)
- Transskribering af Opensky I - exploration (Wulf)

- Slides DALE26W44-API (22 sider, uge 44)
- Bilag med DMI-skærmbillede, lubridate-ark og Mastra-rapport (3 sider med indhold)
- Util.R og OpenSky-script (Caps filer, secret fjernet)